



Smartgolf

Security Assessment

CertiK Assessed on Sept 17th, 2025





CertiK Assessed on Sept 17th, 2025

Smartgolf

The security assessment was prepared by CertiK.

Executive Summary

TYPES

ERC-20

ECOSYSTEM

EVM Compatible

METHODS

Formal Verification, Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 09/17/2025

Final report published on 09/17/2025

Vulnerability Summary



11

Total Findings

0

Resolved

1

Multi-Sig

0

Partially Resolved

10

Acknowledged

0

Declined

2 Centralization

1 Multi-Sig, 1 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

1 Medium

1 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

5 Minor

5 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

3 Informational

3 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | SMARTGOLF

Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Overview](#)

[External Dependencies](#)

[SGiToken](#)

[Privileged Functions](#)

[Owner Role](#)

[DAO Controller Role](#)

[nSGi Contract Role](#)

Findings

[SMA-02 : Initial Token Distribution](#)

[SMA-03 : Centralization Related Risks](#)

[SMA-04 : Clawback Policy Violation: Missing Cliff Period Restriction In `reclaimLockedTokens` Function](#)

[SMA-05 : Missing Zero Address Validation](#)

[SMA-06 : Inconsistent Lock Check Allows Bypass Of Token Locking Mechanism](#)

[SMA-07 : Missing Time Parameter Validation In Token Locking Functions](#)

[SMA-08 : `lockTokens` Function Allows Self-Locking Without Token Transfer](#)

[SMA-11 : Potential Vulnerability Of Gas Exhaustion](#)

[SMA-09 : Functions With `\\_` As Name Prefix Are Not `private` Or `internal`](#)

[SMA-10 : Missing Emit Events](#)

[SMA-12 : Potential Arithmetic Underflow In `\\_availableBalance` Function](#)

Optimizations

[SMA-01 : Code Duplication and Gas Inefficiency in Balance Calculation Functions](#)

Appendix

Disclaimer

CODEBASE | SMARTGOLF

Repository

SmartGolfDAO

Commit

038f0ef3fa1b4e7394b6e19da90f23258b686448

AUDIT SCOPE | SMARTGOLF

SMARTGOLF/SmartGolfDAO



SGi_token.sol

APPROACH & METHODS | SMARTGOLF

This audit was conducted for Smartgolf to evaluate the security and correctness of the smart contracts associated with the Smartgolf project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Formal Verification, Manual Review, and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | SMARTGOLF

Overview

The **SmartGolfDAO** project involves the following smart contract:

- **SGiToken**: This is an ERC20 token contract that implements a token locking and vesting mechanism. The contract allows for the creation of time-locked tokens with configurable cliff periods and vesting schedules. It supports multiple privileged roles for token management, including minting, locking, and reclaiming functionality. The contract integrates with OpenZeppelin's ERC20 and Ownable contracts for standard token functionality and access control.

External Dependencies

The following are external addresses used within the contract:

SGiToken

- `daoController`: Address that can execute DAO-specific functions like locking tokens and minting locked tokens
- `nSgiContract`: Address of the nSGi NFT contract that can mint tokens through the `mintFromNFT` function

We assume these addresses are valid and non-vulnerable actors and implement proper logic to collaborate with the current project.

The following library/contract are considered as the third-party dependencies:

- `@openzeppelin/contracts` (ERC20, Ownable)

Privileged Functions

In the **SmartGolfDAO** project, multiple roles are adopted to ensure the dynamic runtime updates of the project, which were specified in the centralized findings.

Owner Role

The contract owner has authority over the following functions:

- `setDAOController()` : Set the DAO controller address
- `setNSgiContract()` : Set the nSGi contract address
- `transferOwnership()` : Transfer ownership to a new address (inherited from Ownable)
- `renounceOwnership()` : Renounce ownership (inherited from Ownable)

DAO Controller Role

The DAO controller has authority over the following functions:

- `lockTokens()` : Lock tokens for recipients with vesting schedules

- `mintLocked()` : Mint new tokens and lock them with vesting schedules
- `reclaimLockedTokens()` : Reclaim locked tokens from any address

nSGi Contract Role

The nSGi contract has authority over the following function:

- `mintFromNFT()` : Mint tokens when NFTs are burned

The advantage of these privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community. It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private key of any privileged account is compromised, it could lead to devastating consequences for the project, including unlimited token minting and manipulation of the vesting system.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community.

Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of a `TimeLock` contract.

FINDINGS | SMARTGOLF



11
Total Findings

0
Critical

2
Centralization

0
Major

1
Medium

5
Minor

3
Informational

This report has been prepared for Smartgolf to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 11 issues were identified. Leveraging a combination of Formal Verification, Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
SMA-02	Initial Token Distribution	Centralization	Centralization	● 3/5 Multi-Sig
SMA-03	Centralization Related Risks	Centralization	Centralization	● Acknowledged
SMA-04	Clawback Policy Violation: Missing Cliff Period Restriction In <code>reclaimLockedTokens</code> Function	Logical Issue	Medium	● Acknowledged
SMA-05	Missing Zero Address Validation	Volatile Code	Minor	● Acknowledged
SMA-06	Inconsistent Lock Check Allows Bypass Of Token Locking Mechanism	Logical Issue	Minor	● Acknowledged
SMA-07	Missing Time Parameter Validation In Token Locking Functions	Logical Issue	Minor	● Acknowledged
SMA-08	<code>lockTokens</code> Function Allows Self-Locking Without Token Transfer	Logical Issue	Minor	● Acknowledged
SMA-11	Potential Vulnerability Of Gas Exhaustion	Coding Issue	Minor	● Acknowledged
SMA-09	Functions With <code>_</code> As Name Prefix Are Not <code>private</code> Or <code>internal</code>	Coding Style	Informational	● Acknowledged
SMA-10	Missing Emit Events	Coding Style	Informational	● Acknowledged

ID	Title	Category	Severity	Status
SMA-12	Potential Arithmetic Underflow In <code>_availableBalance</code> Function	Logical Issue	Informational	● Acknowledged

SMA-02 | Initial Token Distribution

Category	Severity	Location	Status
Centralization	● Centralization	SGi_token.sol: 29	● 3/5 Multi-Sig

Description

All of the `SGi` tokens (1 billion) are sent to the contract deployer. This is a centralization risk because the deployer can distribute tokens without obtaining the consensus of the community. Any compromise to these addresses may allow a hacker to steal and sell tokens on the market, resulting in severe damage to the project.

As of **09/15/2025**, based on the information provided by Token Holders, the major token holdings are distributed among several addresses as follows:

Address	Amount	Percentage
<u>SmartGolfToken: DAO Treasur - ⅔ Safe Wallet</u>	669,999,001	66.9114%
<u>0x2Bf12c11...3ae02E558</u>	97,000,000	9.6872%
<u>SmartGolfToken: SGi Founder</u>	50,000,000	4.9934%
<u>SmartGolfToken: Team Reserve - ⅔ Safe Wallet</u>	45,000,000	4.4941%
<u>SmartGolfToken: Deployer</u>	41,915,630	4.1860%

Recommendation

It is recommended that the team be transparent regarding the initial token distribution process. The token distribution plan should be published in a public location that the community can access. The team should make efforts to restrict access to the private keys of the deployer account or EOAs. A multi-signature (⅔, ⅔) wallet can be used to prevent a single point of failure due to a private key compromise. Additionally, the team can lock up a portion of tokens, release them with a vesting schedule for long-term success, and deanonymize the project team with a third-party KYC provider to create greater accountability.

Alleviation

[Smartgolf, 09/16/2025]: Thank you for the observation.

We acknowledge that the initial mint allocated the full supply to the deployer.

However, shortly after deployment, 72% of the supply has already been transferred to a 3-of-5 Gnosis Safe multi-sig

Treasury.

The remaining allocations (team reserve, advisor, private sale, liquidity) are also managed through locked allocations with DAO/Owner oversight.

We will publish the Treasury Safe address and its signers in our GitHub policies/registries for transparency.

Future DAO governance with timelock control will further decentralize token management.

Link to the token distribution plan here:

<https://github.com/SMARTGOLF/SmartGolfDAO/blob/main/supply/token-distribution.md>

Multi-sig wallets :

- DAO Treasury Safe: 0xf459f4Ed2caDEda1696a996002F0CEC301DF6E01
- Team Treasury Safe: 0xd2c329a36fd13b9D27f6f17d61c1a5251B4203F5

Signers (DAO Treasury 3/5):

- Signer 1: 0x8f6018181d2750013e756c9363916e46C780FE8
- Signer 2: 0xe9eBA2A57159B34197E0C9Ce61b2d2e6e21B850b
- Signer 3: 0xabf6eaf6e38C480Ef9D12d3462b7BCE43B545aC3
- Signer 4: 0xDCA896738Bd661acf847c31390c3BA06292F1D35
- Signer 5: 0xb4ff769c035cc410e44447206032c2dc7049264b

SMA-03 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	● Centralization	SGi_token.sol: 38, 42, 86, 100, 113, 126	● Acknowledged

Description

SGiToken

Owner

In the contract `SGiToken` the role `Owner` has authority over the functions below.

Direct Owner Functions:

- `setDAOController(address _dao)` - Set the DAO controller address
- `setNSgiContract(address _nSgi)` - Set the nSGi contract address
- `lockTokens(address recipient, uint256 amount, uint256 startTime, uint256 cliff, uint256 duration, string memory lockType)` - Lock tokens for a recipient with vesting schedule
- `mintLocked(address recipient, uint256 amount, uint256 startTime, uint256 cliff, uint256 duration, string memory lockType)` - Mint new tokens and lock them with vesting schedule
- `reclaimLockedTokens(address from, uint index)` - Reclaim locked tokens from any address

Inherited from Ownable:

- `transferOwnership(address newOwner)` (inherited from Ownable)
- `renounceOwnership()` (inherited from Ownable)

Any compromise to the `Owner` account may allow the hacker to take advantage of this authority and:

- **DAO Controller Manipulation:** Change the DAO controller address via `setDAOController()`, potentially redirecting DAO privileges to a malicious address
- **NFT Contract Manipulation:** Change the nSGi contract address via `setNSgiContract()`, allowing malicious contracts to mint tokens via `mintFromNFT()`
- **Ownership Transfer:** Transfer ownership to a malicious address via `transferOwnership()`
- **Complete Control:** Gain complete control over the token contract and all privileged functions
- **Unlimited Minting:** Mint unlimited amounts of SGi tokens to any address via `mintLocked()`, causing severe inflation and devaluing the token
- **Token Locking Manipulation:** Lock tokens for malicious addresses or manipulate vesting schedules via `lockTokens()`
- **Token Theft:** Reclaim locked tokens from legitimate holders via `reclaimLockedTokens()`, effectively stealing their vested tokens

- **Vesting Schedule Abuse:** Create unfair vesting schedules that benefit malicious actors

DAO Controller

In the contract `SGiToken` the role `DAO Controller` (set by Owner) has authority over the functions below.

- `lockTokens(address recipient, uint256 amount, uint256 startTime, uint256 cliff, uint256 duration, string memory lockType)` - Lock tokens for a recipient with vesting schedule
- `mintLocked(address recipient, uint256 amount, uint256 startTime, uint256 cliff, uint256 duration, string memory lockType)` - Mint new tokens and lock them with vesting schedule
- `reclaimLockedTokens(address from, uint index)` - Reclaim locked tokens from any address

Any compromise to the `DAO Controller` account may allow the hacker to take advantage of this authority and:

- **Unlimited Minting:** Mint unlimited amounts of SGi tokens to any address via `mintLocked()`, causing severe inflation and devaluing the token
- **Token Locking Manipulation:** Lock tokens for malicious addresses or manipulate vesting schedules via `lockTokens()`
- **Token Theft:** Reclaim locked tokens from legitimate holders via `reclaimLockedTokens()`, effectively stealing their vested tokens
- **Vesting Schedule Abuse:** Create unfair vesting schedules that benefit malicious actors

nSGi Contract

The `nSGi Contract` (set by Owner) has authority over the function below.

- `mintFromNFT(address to, uint256 amount)` - Mint tokens when NFTs are burned

Any compromise to the `nSGi Contract` may allow the hacker to take advantage of this authority and:

- **Unlimited Minting:** Mint unlimited amounts of SGi tokens to any address via `mintFromNFT()`, causing severe inflation

Risk Assessment

The most critical risks are the **unlimited minting capabilities** through multiple functions. This represents extreme centralization risks because:

1. **Multiple Minting Vectors:** The contract has three different ways to mint tokens:

- `mintLocked()` (`onlyDAO` role)
- `mintFromNFT()` (`nSGi Contract`)

2. **Inflation Attack:** Any compromised privileged account can mint unlimited tokens, completely destroying the token's value

3. **Token Locking Abuse:** The `onlyDAO` role can manipulate the token locking system:

- Lock tokens for malicious addresses
- Create unfair vesting schedules

- Reclaim tokens from legitimate holders

4. **No Supply Cap:** Unlike many tokens that have a fixed supply, this token has no protection against unlimited minting through multiple privileged functions

I Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

I Alleviation

[Smartgolf, 09/16/2025]:

Issue acknowledged.

We recognize that the current implementation grants minting/locking/reclaiming privileges to Owner and DAO Controller roles.

To mitigate the risk, we have already transferred 72% of supply into a 3-of-5 Gnosis Safe multi-sig, and all future privileged operations are executed through the Safe.

Planned improvements:

- Assigning DAO Controller to the Safe, ensuring that any privileged function requires multi-sig approval.
- Introducing a timelock contract (e.g., 48h delay) prior to executing sensitive operations, to provide transparency and awareness.
- Transitioning DAO governance via Snapshot/On-chain voting to progressively decentralize decision making.

Long-term roadmap:

- Once DAO governance is fully in place, ownership will be renounced and Safe/DAO will permanently hold control of all privileged functions.
- This will align with CertiK's recommendation for full decentralization and permanent resolution.

We will also publish the Safe address, signers, and DAO governance information in our GitHub registry for transparency.

[CertiK, 09/17/2025]:

As of 09/17/2025, the **owner** of the `SGiToken` contract is an externally owned account (EOA) with the ability to modify addresses such as `nSgiContract` and `daoController`, as well as mint `SGi` tokens.

It is suggested to implement the aforementioned methods to avoid centralized failure. CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

SMA-04 Clawback Policy Violation: Missing Cliff Period Restriction In `reclaimLockedTokens` Function

Category	Severity	Location	Status
Logical Issue	● Medium	SGi_token.sol: 116	● Acknowledged

Description

The `reclaimLockedTokens` function does not enforce the clawback policy restrictions, specifically the requirement that clawback may only be exercised before the cliff period starts. This creates a conflict between the documented policy and the actual implementation.

```
function reclaimLockedTokens(address from, uint index) external onlyDAO {
    require(index < lockedTokens[from].length, "Invalid index");
    LockInfo memory lock = lockedTokens[from][index];
    _transfer(from, msg.sender, lock.amount);

    for (uint i = index; i < lockedTokens[from].length - 1; i++) {
        lockedTokens[from][i] = lockedTokens[from][i + 1];
    }
    lockedTokens[from].pop();

    emit Reclaimed(from, msg.sender, lock.amount);
}
```

The function poses several risks, including a contradiction with the documented policy by allowing clawback at any time, even after vesting has commenced. Additionally, there is a KYC/AML compliance gap, as the policy specifies clawback for "KYC/AML failure or legal requirements" as an exceptional use case, but the current implementation would fail if such issues are discovered after the cliff period due to insufficient balance (since users may have already spent their vested tokens). Lastly, the function is prone to reversion if the target address has already spent part of their locked tokens, making it unreliable for enforcing the policy effectively.

These vulnerabilities can lead to potential issues, including inconsistency between the implementation and the documented policy, an inability to enforce KYC/AML requirements after the cliff period due to potential insufficient balances, and operational failures where the function could fail when invoked for legitimate policy enforcement.

Recommendation

It's recommended to implement proper policy enforcement in the `reclaimLockedTokens` function and consider implementing a separate emergency clawback function for post-cliff KYC/AML cases that handles partial reclaims appropriately.

I Alleviation

[Smartgolf, 09/16/2025]:

Issue acknowledged.

We recognize that the `reclaimLockedTokens` function does not strictly enforce the cliff restriction at the contract level. Since redeployment would change the token address, we have chosen to mitigate this risk through operational governance and transparency instead.

- A Clawback Policy has been published in our GitHub

(<https://github.com/SMARTGOLF/SmartGolfDAO/blob/main/policies/clawback-policy.md>), which explicitly restricts clawbacks to pre-cliff cases, with the sole exception of KYC/AML failure or legal obligations.

- All clawback actions are logged in the public Clawback Registry

(<https://github.com/SMARTGOLF/SmartGolfDAO/blob/main/registry/clawback-registry.csv>) with date, address, amount, and transaction hash.

- While the function is callable by the DAO/Owner role, operational execution follows the 3-of-5 Gnosis Safe governance model. This prevents unilateral clawbacks even though multi-sig is not enforced at the code level.

- To date, four clawback events have been executed, all documented in the registry and fully aligned with our policy.

We believe this combination of policy + registry + Safe governance sufficiently mitigates the risk without requiring redeployment. Future DAO governance with timelock will further strengthen enforcement.

SMA-05 | Missing Zero Address Validation

Category	Severity	Location	Status
Volatile Code	Minor	SGi_token.sol: 28, 30, 38, 39, 42, 43	Acknowledged

Description

Addresses are not validated before assignment or external calls, potentially allowing the use of zero addresses and leading to unexpected behavior or vulnerabilities. For example, transferring tokens to a zero address can result in a permanent loss of those tokens.

```
39      daoController = _dao;
```

- `_dao` is not zero-checked before being used.

```
30      daoController = _daoController;
```

- `_daoController` is not zero-checked before being used.

```
43      nSgiContract = _nSgi;
```

- `_nSgi` is not zero-checked before being used.

Recommendation

It is recommended to add a zero-check for the passed-in address value to prevent unexpected errors.

Alleviation

[Smartgolf, 09/17/2025]:

Issue acknowledged.

We recognize that the contract does not explicitly check for zero-address values when assigning controller contracts. While this is a valid best practice, we assess the current risk as low because:

- All controller addresses (`daoController`, `_daoController`, `nSgi`) are set during controlled deployment and not updated arbitrarily.
- The addresses are managed by the project team through verified contracts and have been publicly verified on Etherscan.
- There is no user-facing function that would allow arbitrary assignment to the zero address.

As a mitigation, we have documented in our governance and deployment policies that zero-address assignments are not

permitted. Future contract upgrades will include explicit zero-address checks to align with best practices.

Given the current deployment context, no immediate changes will be made, but the issue is noted and planned for future iterations.

SMA-06 Inconsistent Lock Check Allows Bypass Of Token Locking Mechanism

Category	Severity	Location	Status
Logical Issue	Minor	SGi_token.sol: 94	Acknowledged

Description

The contract exhibits inconsistent behavior in checking locked tokens across different functions. While the public `transfer` and `transferFrom` functions properly use `_availableBalance()` to prevent transferring locked tokens, the `lockTokens` function uses `balanceOf()` which does not account for locked tokens.

```
function lockTokens(
    address recipient,
    uint256 amount,
    uint256 startTime,
    uint256 cliff,
    uint256 duration,
    string memory lockType
) external onlyDAO {
    @> require(balanceOf(msg.sender) >= amount, "SGi: insufficient balance for locking");
    _transfer(msg.sender, recipient, amount);
    lockedTokens[recipient].push(LockInfo(amount, startTime, cliff, duration, lockType));
    emit Locked(recipient, startTime, cliff, duration, amount, lockType);
}
```

This inconsistency allows the `onlyDAO` role to bypass the token locking mechanism by transferring locked tokens through the `lockTokens` function.

Recommendation

It's recommended to replace `balanceOf(msg.sender)` with `_availableBalance(msg.sender)` in the `lockTokens` function to maintain consistency with the contract's locking mechanism.

Alleviation

[Smartgolf, 09/16/2025]:

Issue acknowledged.

We recognize that the `lockTokens` function currently checks balances using `balanceOf()` rather than `_availableBalance()`, which could allow the `onlyDAO` role to bypass locked balances.

However, the risk is limited because:

- The function is restricted to the `onlyDAO` role and cannot be called by regular users.
- DAO operations require multi-sig approvals, preventing unilateral misuse.
- All DAO treasury transactions are publicly visible and subject to community review.

As mitigation, we have documented in our governance policies that DAO operations must respect lock constraints, and no locked tokens may be reassigned via this function.

For future contract upgrades, we plan to align the logic by replacing `balanceOf()` with `\_availableBalance()` to maintain consistency across all transfer and locking functions.

SMA-07 | Missing Time Parameter Validation In Token Locking Functions

Category	Severity	Location	Status
Logical Issue	Minor	SGi_token.sol: 86~93, 100~107	Acknowledged

Description

In the `lockTokens` and `mintLocked` functions, the contract does not validate the time parameters (`startTime`, `cliff`, `duration`) before creating lock records. This allows for the creation of invalid or malicious lock configurations that can result in immediate token release or permanent locking.

```
function lockTokens(
    address recipient,
    uint256 amount,
    uint256 startTime,
    uint256 cliff,
    uint256 duration,
    string memory lockType
) external onlyDAO {
    require(balanceOf(msg.sender) >= amount, "SGi: insufficient balance for locking");
    _transfer(msg.sender, recipient, amount);
    lockedTokens[recipient].push(LockInfo(amount, startTime, cliff, duration, lockType));
    emit Locked(recipient, startTime, cliff, duration, amount, lockType);
}

function mintLocked(
    address recipient,
    uint256 amount,
    uint256 startTime,
    uint256 cliff,
    uint256 duration,
    string memory lockType
) external onlyDAO {
    _mint(recipient, amount);
    lockedTokens[recipient].push(LockInfo(amount, startTime, cliff, duration, lockType));
    emit Locked(recipient, startTime, cliff, duration, amount, lockType);
}
```

The `_availableBalance` function relies on these time parameters to calculate available tokens. Invalid time configurations can lead to incorrect balance calculations and unexpected token availability.

Recommendation

It's recommended to add comprehensive validation for time parameters in both functions.

Alleviation

[Smartgolf, 09/16/2025]:

Issue acknowledged.

We recognize that the `lockTokens` and `mintLocked` functions currently do not include explicit validation for time parameters (`startTime`, `cliff`, `duration`). In theory, this could allow invalid lock configurations such as immediate unlocks or indefinite locks.

However, the risk is limited because:

- Both functions are restricted to the `onlyDAO` role and cannot be invoked by regular users.
- DAO actions are executed via multi-sig approval and documented publicly, preventing unilateral or accidental misuse.
- All lock and clawback operations are logged in our GitHub policy/registry for transparency and community oversight.

As mitigation, we have updated internal DAO governance policies to enforce validation rules off-chain before executing any lock operation. Only properly reviewed parameters are submitted to the contract.

For future contract upgrades, we plan to introduce on-chain parameter validation (e.g., nonzero `duration`, logical `cliff < duration`) to fully eliminate this risk.

SMA-08 | lockTokens Function Allows Self-Locking Without Token Transfer

Category	Severity	Location	Status
Logical Issue	Minor	SGi_token.sol: 95	Acknowledged

Description

In the `lockTokens` function, when `msg.sender` and `recipient` are the same address, the `_transfer(msg.sender, recipient, amount)` call does not move any tokens, as both the sender and recipient are identical. Despite the absence of any actual token transfer, a lock record is still created for the specified `amount`. As a result, the internal record of locked tokens does not accurately reflect the recipient's token balance, leading to inconsistencies between the locked amounts and real holdings. This can cause functions such as `getLockedBalance()` and `_availableBalance()` to return inaccurate results or potentially revert during execution due to the state mismatch.

```
function lockTokens(
    address recipient,
    uint256 amount,
    uint256 startTime,
    uint256 cliff,
    uint256 duration,
    string memory lockType
) external onlyDAO {
    require(balanceOf(msg.sender) >= amount, "SGi: insufficient balance for locking");
    _transfer(msg.sender, recipient, amount);
    lockedTokens[recipient].push(LockInfo(amount, startTime, cliff, duration, lockType));
    emit Locked(recipient, startTime, cliff, duration, amount, lockType);
}
```

Recommendation

It's recommended to add a check to prevent self-locking or handle it appropriately.

Alleviation

[Smartgolf, 09/17/2025]:

Issue acknowledged.

The `lockTokens` function may allow self-locking when `msg.sender` and `recipient` are the same, resulting in a lock record without an actual token transfer.

- In practice, this function is restricted to DAO/Owner execution only (not available to end users).

- Our **Lock Policy** (published in GitHub under `/policies/lock-policy.md`) explicitly prohibits self-locking operations. DAO proposals including `recipient == sender` are invalid and will not be executed.
- Any lock operation must be documented in the **Lock Registry** with recipient, amount, and parameters, providing transparency and detection of invalid configurations.
- As a result, even if the contract does not block this edge case, governance and registry requirements ensure it cannot be abused in practice.

We will continue to monitor this in DAO execution, and may introduce additional safeguards (off-chain or via DAO-controlled timelock contracts) if necessary in future versions.

SMA-11 | Potential Vulnerability Of Gas Exhaustion

Category	Severity	Location	Status
Coding Issue	● Minor	SGi_token.sol: 59, 74, 96, 109	● Acknowledged

Description

The contract contains functions vulnerable to gas exhaustion due to unbounded loops and unrestricted input sizes, which could lead to transaction failures and denial of service (DoS) scenarios.

```
function _availableBalance(address account) public view returns (uint256) {
    uint256 locked = 0;
    LockInfo[] storage locks = lockedTokens[account];
    for (uint256 i = 0; i < locks.length; i++) {
        LockInfo storage lock = locks[i];
        if (block.timestamp < lock.startTime + lock.cliff) {
            locked += lock.amount;
        } else if (block.timestamp < lock.startTime + lock.cliff +
lock.duration) {
            uint256 elapsed = block.timestamp - (lock.startTime + lock.cliff);
            uint256 vested = (lock.amount * elapsed) / lock.duration;
            locked += (lock.amount - vested);
        }
    }
    return balanceOf(account) - locked;
}

function getLockedBalance(address account) external view returns (uint256
totalLocked) {
    LockInfo[] storage locks = lockedTokens[account];
    for (uint256 i = 0; i < locks.length; i++) {
        LockInfo storage lock = locks[i];
        if (block.timestamp < lock.startTime + lock.cliff) {
            totalLocked += lock.amount;
        } else if (block.timestamp < lock.startTime + lock.cliff +
lock.duration) {
            uint256 elapsed = block.timestamp - (lock.startTime + lock.cliff);
            uint256 vested = (lock.amount * elapsed) / lock.duration;
            totalLocked += (lock.amount - vested);
        }
    }
}
```

Every call to `transfer` or `transferFrom` triggers `_availableBalance`, which loops through all lock records for the sender. If an account accumulates a large number of lock records, these functions can exceed the block gas limit, causing

transaction failures.

In addition, there is also unrestricted string size for `lockType` in both `lockTokens` and `mintLocked` functions. If an account has many lock records and each includes a large `lockType` string, the cumulative gas cost for iterating through all locks and accessing their `lockType` fields can become significant.

Recommendation

It is recommended to impose a limit on the number of lock records per account and to restrict the size of the `lockType` string to a defined range.

Alleviation

[Smartgolf, 09/17/2025]:

Issue acknowledged.

We understand that looping through a very large number of lock records may increase gas costs.

However, in practice, the number of lock entries per address is intentionally limited by project policy and distribution design.

- Allocations are managed centrally (team, advisors, investors), and normal users will not accumulate unbounded locks.
- DAO Treasury and Team Treasury allocations are aggregated in bulk, not split into excessive individual locks.
- Lock types are predefined, not arbitrary unbounded strings.

This means the risk of gas exhaustion is theoretical and not reachable in normal operations.

We will document these operational limits in our GitHub **Lock Policy** and may consider adding explicit caps in future upgrades.

SMA-09 Functions With `_` As Name Prefix Are Not `private` Or `internal`

Category	Severity	Location	Status
Coding Style	● Informational	SGi_token.sol: 56	● Acknowledged

Description

Functions with names starting with `_` should be declared as `private` / `internal`.

Recommendation

Consider changing function visibility to `private` or removing `_` from the start of the function name.

Alleviation

[Smartgolf, 09/17/2025]:

Issue acknowledged.

This is a coding-style recommendation, not a security concern.

The `_` prefix was used as a naming convention, while the functions are intentionally declared as `public` / `external` for usability.

There is no functional or security risk, and we do not plan to modify the naming convention at this stage.

SMA-10 | Missing Emit Events

Category	Severity	Location	Status
Coding Style	● Informational	SGi_token.sol: 38, 42	● Acknowledged

Description

There should always be events emitted in sensitive functions that are controlled by centralization roles.

Recommendation

It is recommended to emit events in sensitive functions that are controlled by centralization roles.

Alleviation

[Smartgolf, 09/17/2025]:

Issue acknowledged.

We recognize that additional emit events could improve transparency in sensitive functions.

However, this is not a security risk and current logic is unaffected.

For operational clarity, all clawback actions are already logged and documented in our **Clawback Registry** on GitHub.

We may consider adding extra events in future upgrades, but no functional changes are planned for this version.

SMA-12 | Potential Arithmetic Underflow In `_availableBalance` Function

Category	Severity	Location	Status
Logical Issue	● Informational	SGi_token.sol: 69	● Acknowledged

Description

The `_availableBalance` function performs arithmetic subtraction without checking for underflow conditions, which could cause the function to revert in certain edge cases.

```
function _availableBalance(address account) public view returns (uint256) {
    uint256 locked = 0;
    LockInfo[] storage locks = lockedTokens[account];
    for (uint256 i = 0; i < locks.length; i++) {
        LockInfo storage lock = locks[i];
        if (block.timestamp < lock.startTime + lock.cliff) {
            locked += lock.amount;
        } else if (block.timestamp < lock.startTime + lock.cliff +
lock.duration) {
            uint256 elapsed = block.timestamp - (lock.startTime + lock.cliff);
            uint256 vested = (lock.amount * elapsed) / lock.duration;
            locked += (lock.amount - vested);
        }
    }
    return balanceOf(account) - locked;
}
```

Under normal circumstances, the vesting mechanism ensures that `locked` should never exceed `balanceOf(account)` because:

1. When tokens are locked, the total locked amount equals the account balance
2. Over time, the locked amount decreases as tokens vest
3. Users can only transfer their available (unvested) tokens

However, underflow can occur in edge cases where the lock records become inconsistent with the actual balance, for example, when the `lockTokens` function is used to transfer locked tokens (allowing DAO Controller to transfer their own locked tokens).

Recommendation

It's recommended to add underflow protection to handle edge cases gracefully.

Alleviation

[Smartgolf, 09/17/2025]:

Issue acknowledged.

We note the concern regarding potential arithmetic underflow in `_availableBalance`.

However, since the contract is compiled with **Solidity ^0.8.0**, arithmetic underflow/overflow is automatically checked and causes a revert, not a silent wraparound.

In practice:

- The locked amount can never exceed the actual balance of the account, due to enforced checks in `transfer` and `transferFrom`.
- Vesting reduces locked tokens over time, further lowering risk.
- Any inconsistent lock state would revert safely under Solidity 0.8.x rules.

Therefore, the identified risk is **theoretical only**, and no exploitable condition exists.

We will add clarification about this Solidity safeguard in our GitHub documentation to reassure external reviewers.

OPTIMIZATIONS | SMARTGOLF

ID	Title	Category	Severity	Status
SMA-01	Code Duplication And Gas Inefficiency In Balance Calculation Functions	Code Optimization	Optimization	● Acknowledged

SMA-01 | Code Duplication And Gas Inefficiency In Balance Calculation Functions

Category	Severity	Location	Status
Code Optimization	● Optimization	SGi_token.sol: 56, 72	● Acknowledged

Description

The `_availableBalance` and `getLockedBalance` functions both implement identical vesting logic, leading to unnecessary code duplication. Additionally, both functions access vesting data using `LockInfo storage lock = locks[i]`, which incurs higher gas costs because data is read from storage on every iteration, even though the functions only perform read-only operations. Refactoring the code to consolidate the vesting logic into a single internal function and utilizing memory arrays for read-only loops would help reduce gas usage and improve maintainability.

```
function _availableBalance(address account) public view returns (uint256) {
    uint256 locked = 0;
    LockInfo[] storage locks = lockedTokens[account];
    for (uint256 i = 0; i < locks.length; i++) {
        LockInfo storage lock = locks[i];
        if (block.timestamp < lock.startTime + lock.cliff) {
            locked += lock.amount;
        } else if (block.timestamp < lock.startTime + lock.cliff +
lock.duration) {
            uint256 elapsed = block.timestamp - (lock.startTime + lock.cliff);
            uint256 vested = (lock.amount * elapsed) / lock.duration;
            locked += (lock.amount - vested);
        }
    }
    return balanceOf(account) - locked;
}

function getLockedBalance(address account) external view returns (uint256
totalLocked) {
    LockInfo[] storage locks = lockedTokens[account];
    for (uint256 i = 0; i < locks.length; i++) {
        LockInfo storage lock = locks[i];
        if (block.timestamp < lock.startTime + lock.cliff) {
            totalLocked += lock.amount;
        } else if (block.timestamp < lock.startTime + lock.cliff +
lock.duration) {
            uint256 elapsed = block.timestamp - (lock.startTime + lock.cliff);
            uint256 vested = (lock.amount * elapsed) / lock.duration;
            totalLocked += (lock.amount - vested);
        }
    }
}
```

Recommendation

Refactor the code to eliminate duplication and improve gas efficiency.

Alleviation

[Smartgolf, 09/17/2025]:

Issue acknowledged.

This is a code-quality/gas-optimization recommendation, not a security risk.

The affected logic is used in **view/bounded DAO workflows** with low call frequency, and our Lock Policy limits the number of lock entries per address. Therefore, the current implementation has **no practical performance or cost impact** on users or normal operations.

We will keep this item as **low-priority** and may refactor later (helper function + minor micro-optimizations) without changing behavior, but **no changes are planned for this version**.

APPENDIX | SMARTGOLF

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

